

Documentación del Sistema Pokémon

Guía Técnica de Implementación Java + MySQL

1. Configuración de la Base de Datos (MySQL)

Se definió una estructura relacional con dos tablas principales:

pokemon

y

mochila

.

Script de Creación

```
CREATE DATABASE sistema_po;
USE sistema_po;

CREATE TABLE pokemon (
    i_pokemon INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    tipo VARCHAR(30) NOT NULL,
    nivel INT,
    maximos INT,
    ps_maximos INT
);

CREATE TABLE mochila (
    id_item INT AUTO_INCREMENT PRIMARY KEY,
    nombre_item VARCHAR(50) NOT NULL,
```

```
cantidad INT,  
argumento VARCHAR(50)  
);
```

Corrección de Error: En el script original se intentaba insertar en una columna llamada

```
descripcion
```

, pero la tabla se creó con la columna

```
argumento
```

. Se estandarizó a

```
argumento
```

para evitar el error 1054.

2. Conexión a la Base de Datos (Conexion.java)

Clase encargada de gestionar el Driver JDBC y establecer el puente con el servidor local.

```
package crud_2B;  
import java.sql.Connection;  
import java.sql.DriverManager;  
import java.sql.SQLException;  
  
public class Conexion {  
    public static Connection getConnection() {  
        Connection cnx = null;  
        String url = "jdbc:mysql://localhost:3306/sistema_po";  
        String usr = "root";  
        String psd = "jptam050101";  
        try {  
            Class.forName("com.mysql.cj.jdbc.Driver");
```

```

        cnx = DriverManager.getConnection(url, usr, psd);
    } catch (ClassNotFoundException | SQLException e) {
        System.out.println("Error en conexión: " + e.getMessage());
    }
    return cnx;
}
}

```

3. Modelo de Datos (pokemon.java)

Representación en objetos de la tabla de la base de datos.

```

package crud_2B;

public class pokemon {
    private int id;
    private String nombre, tipo;
    private int nivel, psMaximos;

    public pokemon(String nombre, String tipo, int nivel, int psMaximos) {
        this.nombre = nombre;
        this.tipo = tipo;
        this.nivel = nivel;
        this.psMaximos = psMaximos;
    }
    // Getters...
}

```

4. Lógica de Negocio (CRUD.java)

Implementación de las operaciones fundamentales:

- **Create:** Método

```
agregar(pokemon p)
```

.

- **Read:** Método

```
leer()
```

para mostrar registros en consola.

- **Update:** Método

```
actualizar(id, nivel)
```

.

- **Delete:** Método

```
eliminar(id)
```

.

5. Resolución de Errores Comunes

- **Case Sensitivity:** Se corrigió el error donde se intentaba instanciar

```
clasepokemon
```

cuando la clase real es

```
pokemon
```

(minúsculas).

- **Tipado de Parámetros:** Se eliminó el uso incorrecto de brackets angulares

```
<CRUD>
```

en la declaración de variables locales en el Main.

- **Librerías Externas:** Es obligatorio añadir el

```
mysql-connector-j.jar
```

a las dependencias del proyecto en IntelliJ/NetBeans.